

온프레미스 LLM 추론 엔진 선택 & 평가 체계

llama.cpp vs Ollama 벤치마크 · 2×2 Ablation · QLoRA 프롬프트 증류



2,660회

벤치마크 채점
133케이스×12카테고리×4셀×R5

+21.6pp

검증 레이어 결함 적발·수정
A2 54.3% → 75.9% (CI 비중첩)

~11.7s→2.4s

llama.cpp CUDA + reasoning-off
disambiguation 경로 latency

49% → 96%

프롬프트 증류(QLoRA SFT)
4줄 최소 프롬프트 tool-routing

추론 백엔드 교체가 던진 4개 질문

1. 어떤 Provider가 더 안정적으로 Tool Calling을 수행하는가?

2. JSON 구조화 출력은 얼마나 안정적인가?

3. Validation Layer는 실제로 도움이 되는가?

4. Fine-tuning이 정말 필요한 상황인가?

핵심 철학: 단순 latency 비교로는 답할 수 없다.
"모델 성능"과 "시스템(후처리 레이어) 성능"을 분리해
측정해야 엉뚱한 레이어를 고치지 않는다.

규모 133 케이스 × 12 카테고리 × R5 × 4셀 = 2,660회 채점
(재시도 포함 ~2,860 LLM 호출) · 전 항목 Wilson 95% CI

2×2 Ablation — 한 변수만 분리

A1

Ollama · Layer OFF
(intrinsic)

A2

Ollama · Layer ON
(production)

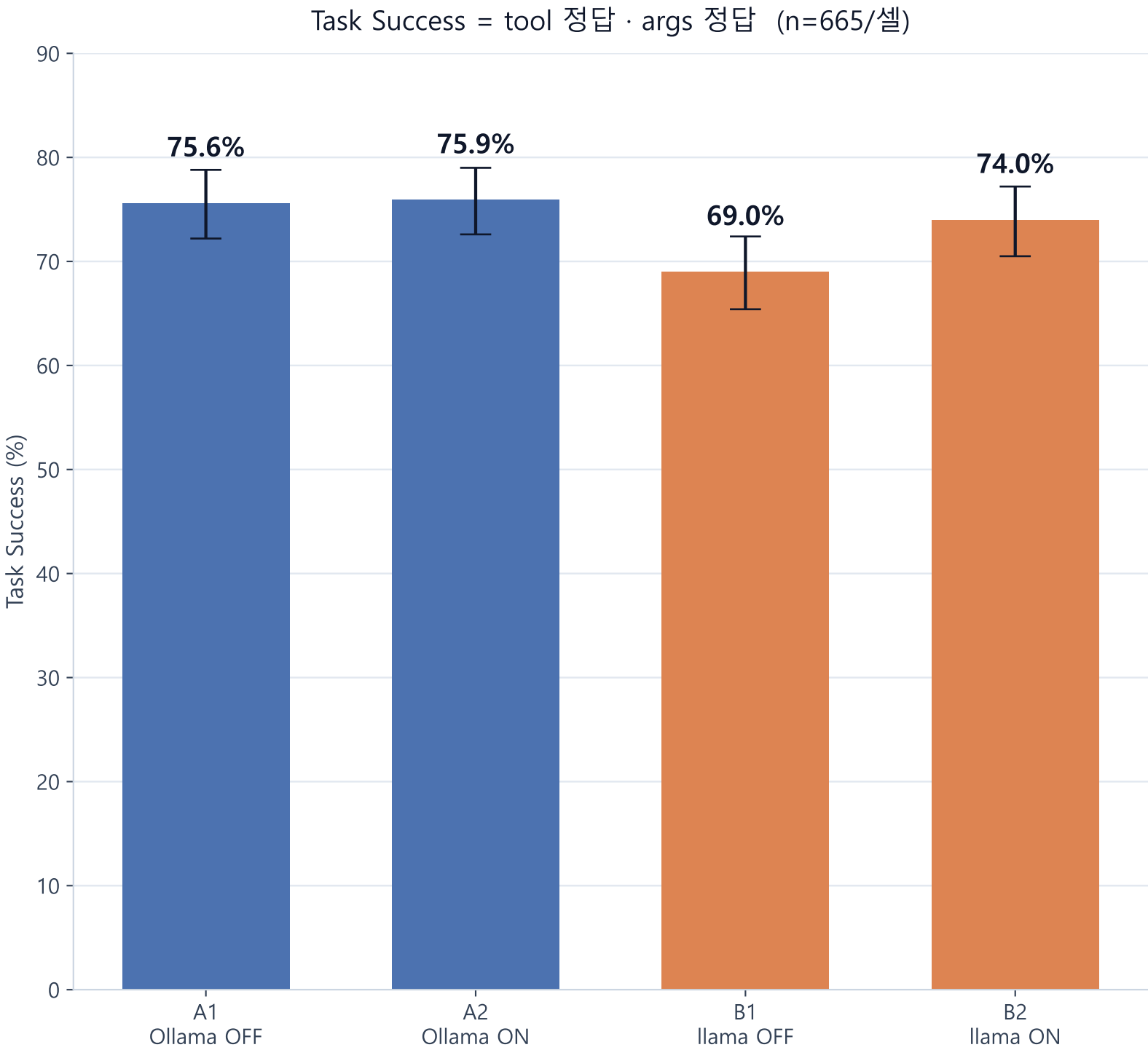
B1

llama.cpp · Layer OFF
(intrinsic)

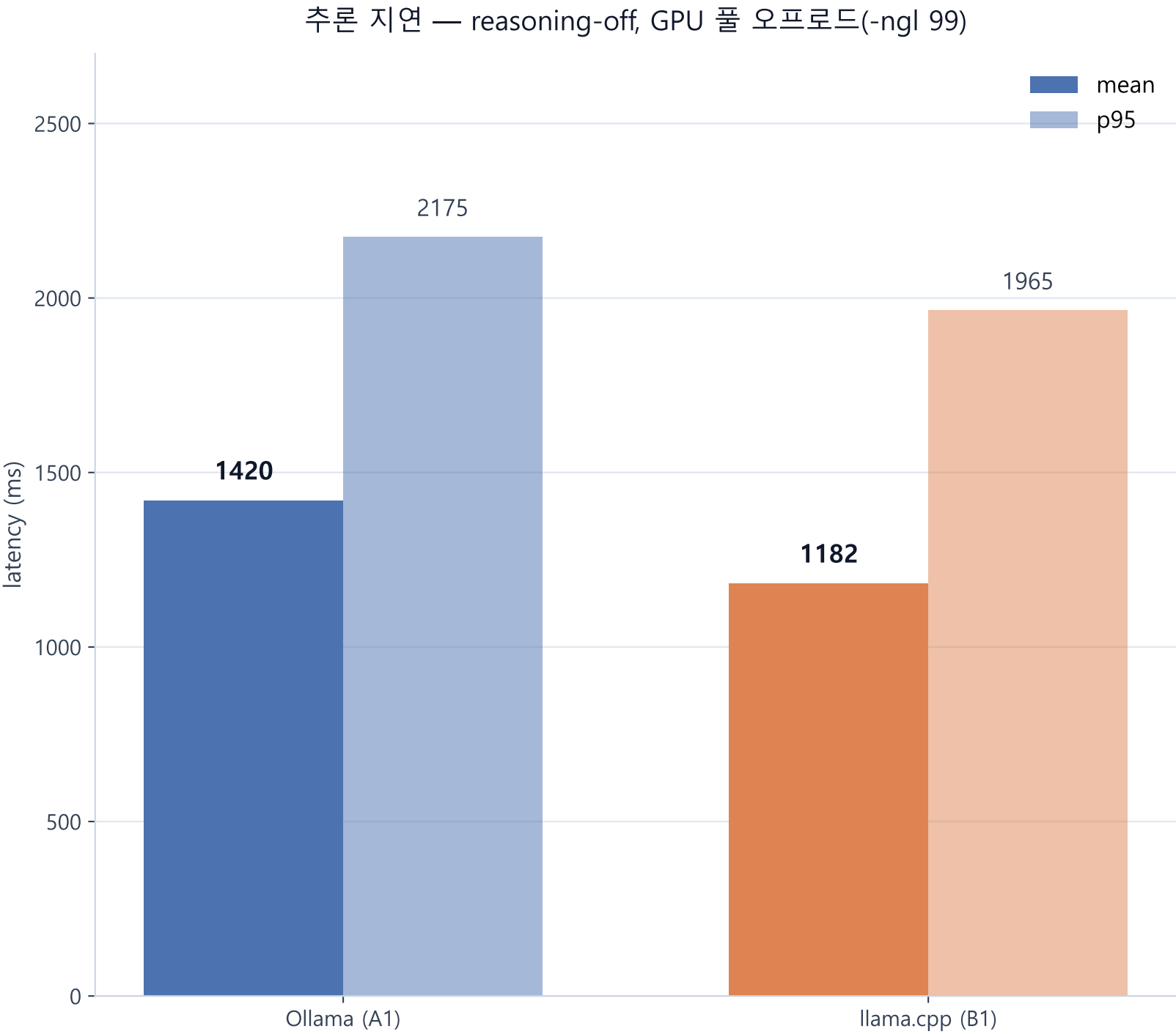
B2

llama.cpp · Layer ON
(production)

LlamaCppLlmGateway가 OllamaLlmGateway를 상속해 transport만 오버라이드
→ 레이어 로직이 두 Provider에 바이트 단위로 동일 → 순수 비교 성립

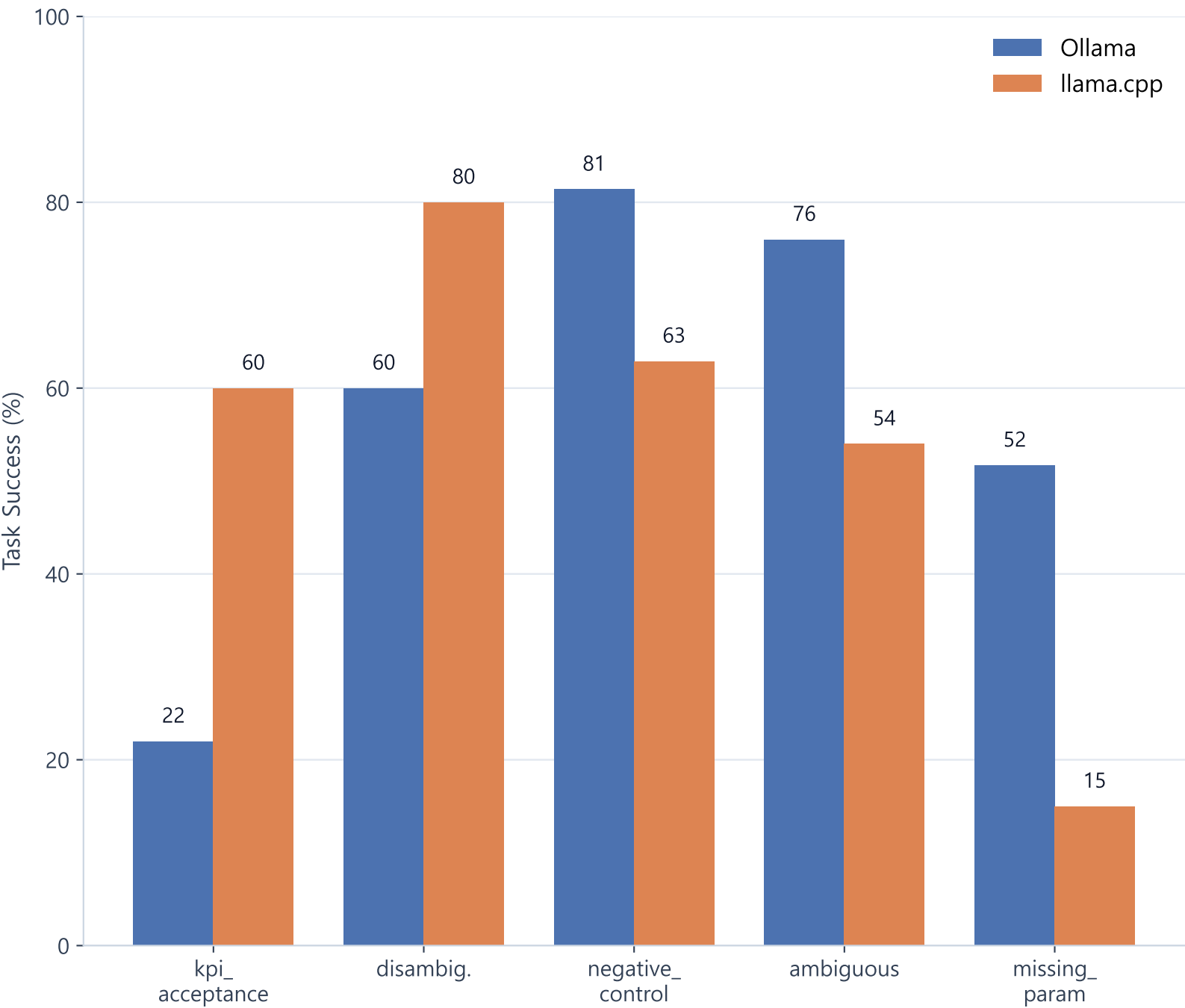


A1 = A2 CI 완전 중첩 → 레이어는 Ollama에 통계적 중립 · B1 vs B2 비중첩 → llama.cpp엔 진짜 +5.0pp



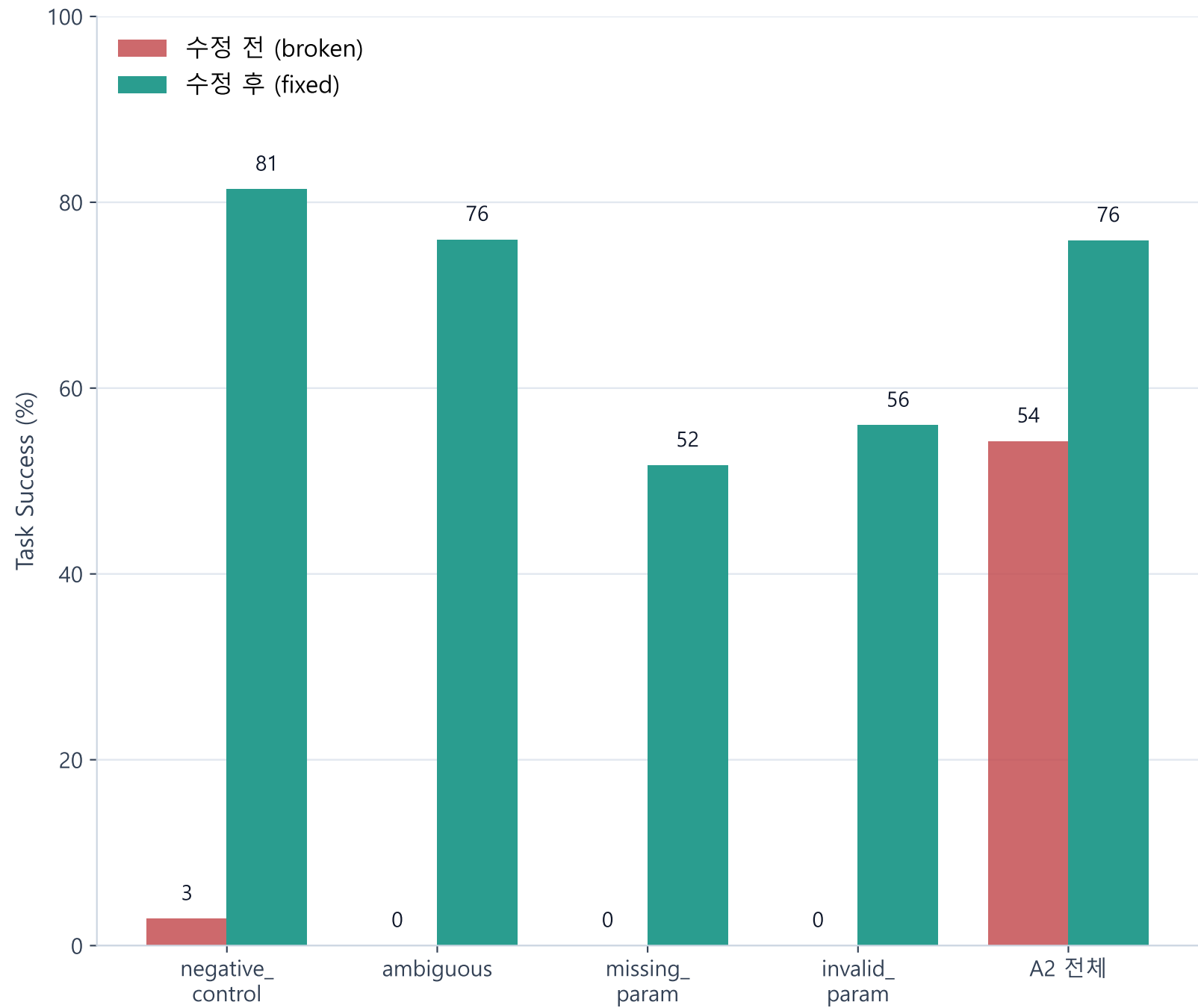
둘 다 reasoning-off(thinking 차이 아님) → ~240ms 격차는 서버 오버헤드 · llama.cpp mean 17% ↓

카테고리별 강·약점 — 명확한 트레이드오프



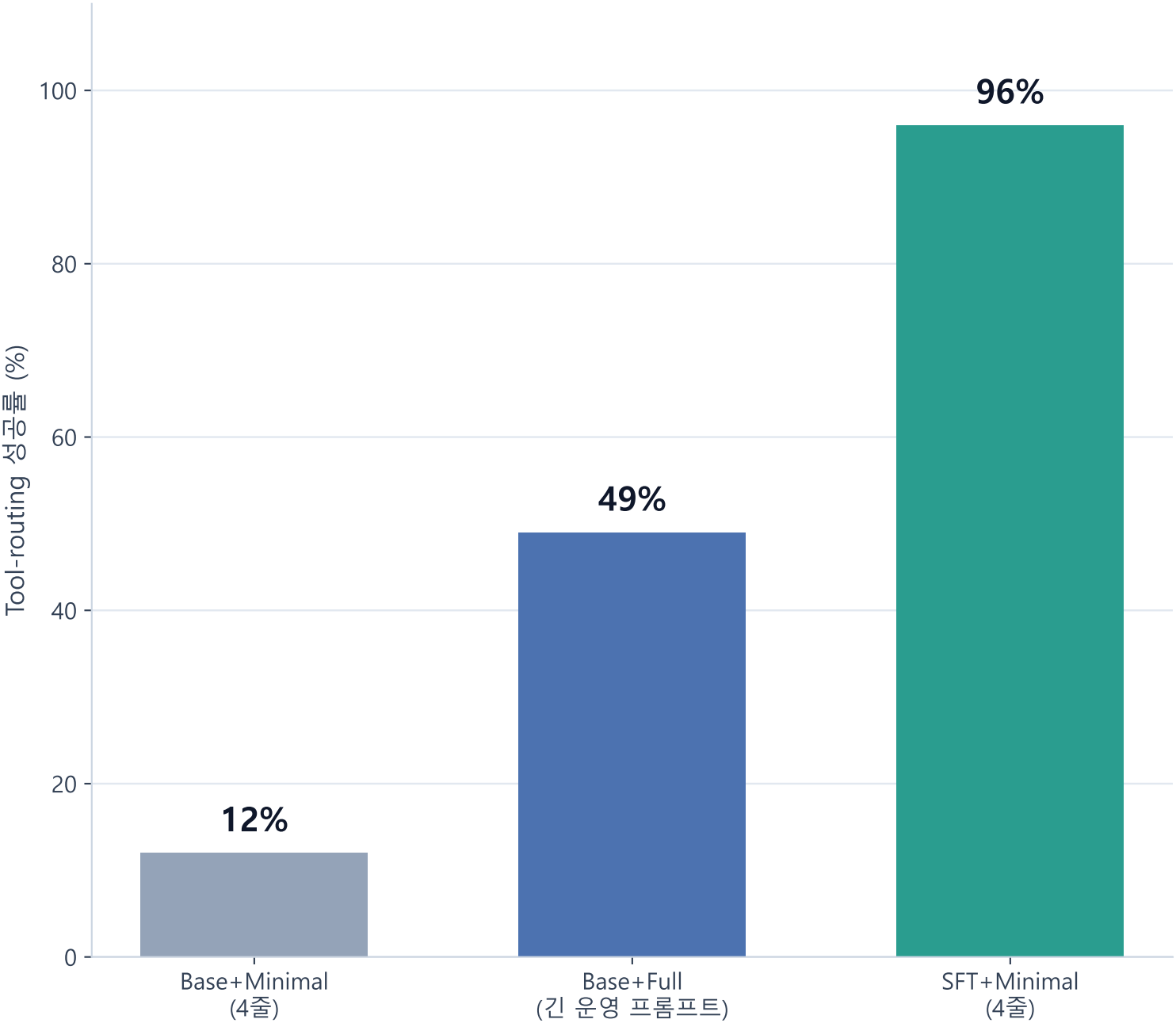
llama.cpp 우위: 추출형(kpi-disambig) ↔ Ollama 우위: 거절형(negative-ambiguous-missing)
원인: llama.cpp 경로의 tool_choice:"auto" 편향 + 채팅 템플릿/샘플링 기본값 차이 (코드 레벨 규명)

운영 레이어가 최고 모델을 -21pp 악화 → 수정

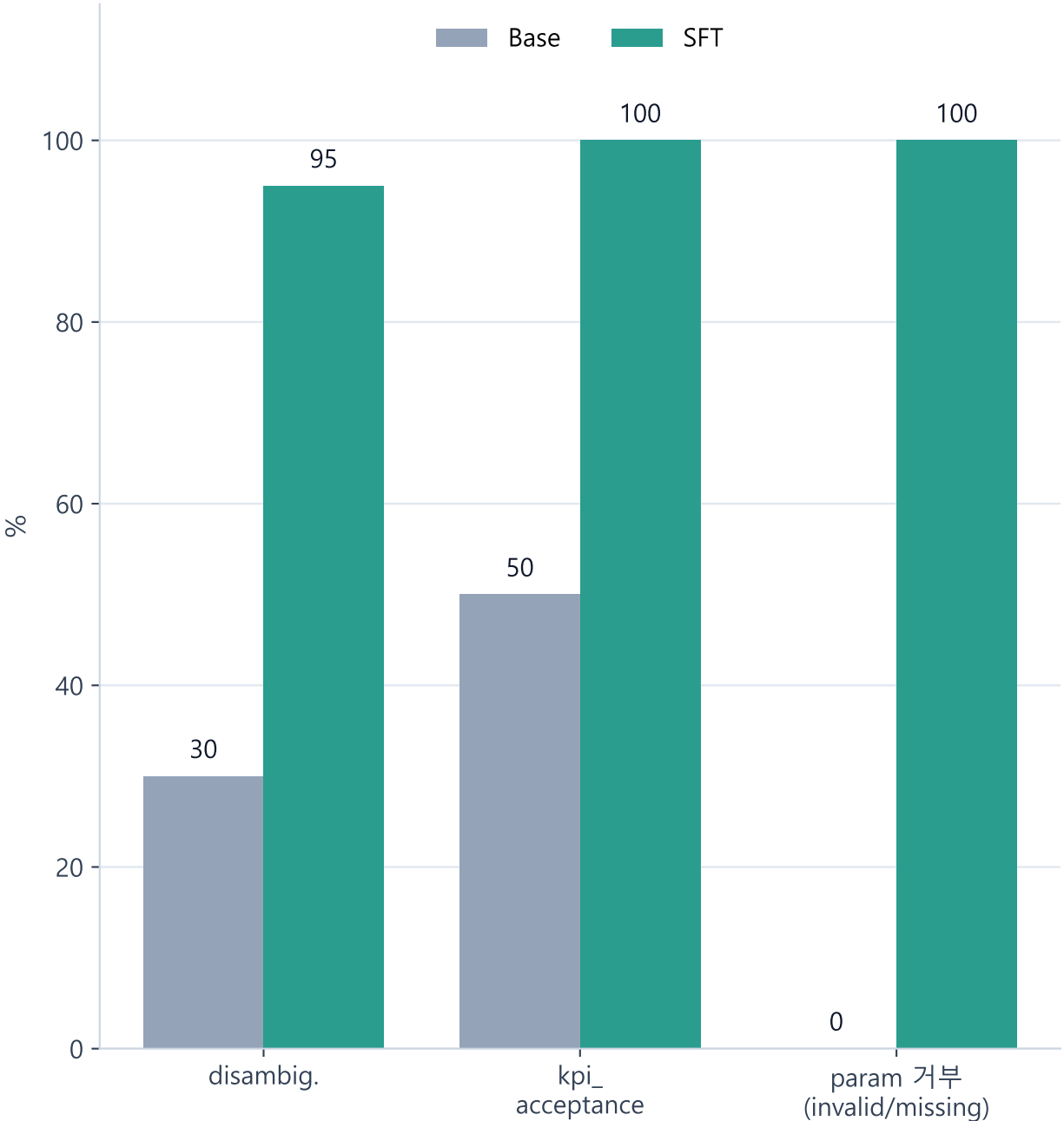


Repair-retry가 거부(decline)를 강제로 tool 환각으로 변환 + Validator 범위검사 부재
→ decline 중단 인정 + range-check 추가 · A2 54.3% → 75.9% (Wilson CI 비중첩, +21.6pp)

3-조건 held-out 평가 (n=100, 동일 서버)



핵심 카테고리 (4줄 프롬프트)



왜 증류인가

역량이 가중치가 아니라
긴 프롬프트 문자열에 있었다
→ 유지보수·확장·추론비용 부채

데이터 / 학습

450행(300/50/100), 9개 실제
tool에 grounding, ToolRouter로
450/450 검증 · 라벨 노이즈 0
QLoRA r16/α32, NF4, 0.58%
RTX 4060 Ti 8GB ~7분
eval-loss 0.0315 → 0.0029

배포

merge→GGUF(q4_k_m 1.27GB)
→ 베이스와 동일 런타임/플래그

SFT+Minimal(96%) = Base+Full(49%)의 약 2배, Base+Minimal(12%)의 8배 — 라우팅이 프롬프트가 아닌 가중치에 · 프롬프트 의존도: 베이스 49→12(-37pp) 붕괴, SFT는 96 유지

Fine-tuning 게이트 (실행 전 동결)

- 레이어 효과 미미 (A2 = A1)**
베이스 충분 → fine-tune 생략, 프롬프트/스키마에 투자
- lift 대부분이 fallback**
제품이 결정론적 코드에 업혀있음 → 정직히 문서화
- 1차 스키마 ↓ & fallback이 하드 카테고리 못 덮음**
fine-tune 정당화 → Phase 3

실제 결과: 첫 분기 발동(A1 = A2). 수정 후 JSON parse 100% / 1차 스키마 94% 도달
→ "포맷 병목" 가설 사망 → 정확도 목적의 fine-tuning 근거 무효화.

남은 의미론적 잔여 2개를 가장 싼 레버로 해소:

- kpi_acceptance — 평문 2줄 추가 → 22% → 94%
- disambiguation — 서빙 플래그(reasoning-off) → 63% → 91.7%

결론 — 엔지니어링 판단의 방어 가능성

-21pp 적발

운영 레이어가 최고 모델을 악화시킴
2,660회 + Wilson CI로 적발·수정 (54.3→75.9%)

무료 레버 우선

"약하니 fine-tune" 본능을 프롬프트 2줄
(KPI 22→94%)·서빙 플래그(63→91.7%)로 무효화

증류로 재정의

SFT 목표를 정확도가 아닌 프롬프트 의존 제거로
재정의 → 49→96%, 의존 -37pp 제거

측정이 본능을 이긴다 · 가장 싼 레버부터 · 모델≠시스템